

Celem mojego projektu jest stworzenie prostej aplikacji internetowej, umożliwiającej przeliczanie walut. Podzieliłam go na część frontendową i backendową.

Część frontendowa składa się ze strony internetowej napisanej w HTML, CSS i JS. Backend natomiast opiera się na mikroserwisie API napisanym w Pythonie z wykorzystaniem frameworka Flask. Całość uruchomiłam w środowisku Docker, co ułatwia jego konfigurację i uruchamianie aplikacji na różnych urządzeniach.

Aplikacja ma umożliwiać użytkownikowi szybko i prosto przeliczać wybraną kwotę na podstawie ustalonego kursu walutowego. Projekt ma za zadanie pokazać praktyczne zastosowanie mikroserwisów oraz współpracę między frontendem a backendem.

Docker

Docker jest platformą umożliwiającą uruchomienie aplikacji w kontenerach - środowiskach, które zawierają tylko potrzebne pliki do uruchomienia danej aplikacji, przez co są bardzo lekkie i pozwalają na dużo mniejsze zużycie zasobów (CPU, RAM) w porównaniu z wirtualnymi maszynami.

Docker wykorzystuje obrazy, czyli gotowe „szablony” zawierające aplikację i jej konfigurację, z których uruchamiane są kontenery. W moim projekcie Docker służy do uruchamiania backendu Flask oraz frontendu, gwarantując identyczne działanie aplikacji na każdym komputerze.

Docker Compose

Jest to narzędzie służące do uruchamiania wielu kontenerów jednocześnie, za pomocą jednego pliku YAML. Możemy opisać w nim działanie naszej całej aplikacji, ustalić porty oraz zależności pomiędzy kontenerami. Nie musimy uruchamiać każdego konteneru z osobna - możemy zrobić to jednym poleceniem, spajając w całość nasze wszystkie usługi. W moim projekcie Docker Compose koordynuje uruchomienie backendu i frontendu, zapewniając ich poprawną komunikację i spójne działanie.

Mikroserwis

//**

Cytat zaczerpnięty ze strony

<https://martinfowler.com/articles/microservices.html>

*Styl architektury mikroserwisowej to podejście do tworzenia jednej aplikacji jako zestawu małych usług, z których każda działa w osobnym procesie i komunikuje się za pomocą lekkich mechanizmów, często poprzez API zasobów HTTP. **//*

W praktyce oznacza to, że zamiast budować jedną dużą i trudną w utrzymaniu aplikację, dzieli się ją na wiele mniejszych elementów, z których każdy ma jasno określone zadanie. Dzięki temu łatwiej jest je rozwijać, testować oraz wdrażać, a także szybciej reagować na błędy. Taki podział zwiększa przejrzystość projektu i ułatwia zarządzanie nim na dłuższą metę.

Flask

Flask jest frameworkiem pozwalającym tworzyć aplikacje webowe oraz API w prosty sposób. Dzięki flask, aplikacja może reagować na konkretne adresy URL. W moim projekcie obsługuje on logikę przeliczania walut oraz zwracanie danych w formacie JSON. Jest często wykorzystywany w mikroserwisach ze względu na lekkość i elastyczność. Pozwala także łatwo rozszerzać funkcjonalność o dodatkowe moduły.

HTML

HTML jest standardowym językiem znacznikowym, który służy do tworzenia stron internetowych. Definiuje on strukturę i zawartość strony, np. nagłówki, przyciski, formularze za pomocą znaczników. Przeglądarka odczytuje te znaczniki i na ich podstawie wyświetla zawartość w czytelnej formie. W moim projekcie HTML stanowi podstawę interfejsu użytkownika. Umożliwiają on wpisanie kwoty oraz wybór walut. HTML pozwala również zastosować JavaScript i CSS, dzięki czemu strona jest dynamiczna i lepiej wygląda.

JSON

JSON to lekki format wymiany danych, który jest łatwy do odczytania i zapisania. Służy do przesyłania informacji między frontendem a backendem w aplikacjach webowych. Flask zwraca wyniki konwersji w formacie JSON, dzięki czemu JavaScript może je łatwo przetworzyć i wyświetlić. JSON jest uniwersalny i wspierany w większości języków programowania. Pozwala utrzymać przejrzysty i uporządkowany sposób komunikacji między usługami.

W moim projekcie JSON pozwala backendowi przekazywać dane o kwocie, walucie wejściowej i docelowej oraz wyniku. Dzięki temu frontend może odczytać te dane i dynamicznie wyświetlić wynik na stronie, bez konieczności przeładowywania całej strony. JSON umożliwia też łatwe rozszerzanie

funkcjonalności w przyszłości np. dodanie kolejnych walut, bez konieczności zmian w sposobie komunikacji.

Wybór zastosowanych narzędzi, technologii i środowiska

Do realizacji projektu zdecydowałam się wykorzystać narzędzia i technologie, które były lekkie oraz proste w konfiguracji i które dobrze współpracują ze w środowisku webowym.

HTML, CSS i JavaScript wybrałam do stworzenia interfejsu użytkownika. HTML umożliwia zbudowanie struktury strony, CSS odpowiada za wygląd, a JavaScript wprowadza interaktywność i daje komunikatywność frontendowi z backendem. Dzięki temu użytkownik może wpisywać kwoty, wybierać waluty i natychmiast otrzymywać wynik przeliczenia bez potrzeby przeładowywania strony.

Flask wybrałam jako framework do obsługi backendu. Dzięki niemu aplikacja obsługuje przeliczenia walut i zwraca dane w formacie JSON, które frontend może od razu przetworzyć i wyświetlić.

JSON - wbrałam go ze względu na to, że jest prosty, uniwersalny i dobrze wspiera komunikację między usługami. Dzięki niemu informacje o kwocie, walucie źródłowej i docelowej oraz wyniku konwersji są prawidłowo przesyłane w sposób jednoznaczny.

Docker i **Docker Compose** wybrałam do uruchamiania aplikacji w kontenerach. Docker pozwala odizolować backend w lekkim środowisku, co ułatwia instalację i konfigurację oraz zapewnia identyczne działanie na różnych maszynach. Docker Compose umożliwia natomiast uruchomienie wszystkich usług projektu jednym poleceniem, co znacznie upraszcza testowanie i wdrażanie.

Wszystkie narzędzia zostały wybrane ponieważ są proste w użyciu i łatwo dostępne ale jednocześnie pozwalają tworzyć zaawansowane aplikacje i systemy.

Środowisko i użyte urządzenia

Projekt zrealizowałam na komputerze z systemem Windows 11. Do pisania kodu użyłam Visual Studio Code - edytora z obsługą potrzebnych mi języków programowania - Python, HTML, CSS, JS. Do uruchomienia backendu Flask wykorzystałam **Docker Desktop**.

Frontend działa lokalnie w przeglądarce internetowej (testowałam w Google Chrome. Dzięki temu mogłam sprawdzić poprawność działania aplikacji w czasie rzeczywistym.

Testy

Sprawdzenie, czy aplikacja poprawnie zbuduje się bez błędów.

Aplikacja powinna zostać poprawnie zbudowana za pomocą polecenia `docker compose build --no-cache`. W terminalu nie powinny pojawić się błędy, a proces powinien zakończyć się komunikatem o pomyślnym zbudowaniu obrazów.

```
PS C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt> docker compose build --no-cache
time="2025-12-14T14:47:55+01:00" level=warning msg="C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Building 5/5 (20/20) FINISHED
-> [internal] load local bake definitions
-> reading from stdin 1.21kB
-> [backend internal] load build definition from Dockerfile
-> transferring dockerfile: 194B
-> [frontend internal] load build definition from Dockerfile
-> transferring dockerfile: 106B
-> [backend internal] load metadata for docker.io/library/python:3.12-slim
-> [frontend internal] load metadata for docker.io/library/nginx:alpine
-> [backend internal] load .dockerignore
-> transferring context: 2B
-> [frontend internal] load .dockerignore
-> transferring context: 2B
-> [backend 1/5] FROM docker.io/library/python:3.12-slim@sha256:fa48ee2146644c2388b909d6b7651a768178f84fc9558dc495e4d6d84d01
-> resolve docker.io/library/python:3.12-slim@sha256:fa48ee2146644c2388b909d6b7651a768178f84fc9558dc495e4d6d84d01
-> [backend internal] load build context
-> transferring context: 63B
-> [frontend internal] load build context
-> transferring context: 32B
-> CACHED [frontend 1/2] FROM docker.io/library/nginx:alpine@sha256:052b75ab72f690f33debaa51c7e88d9b969a0447a133eb2b99c985d9188cb2b
-> resolve docker.io/library/nginx:alpine@sha256:052b75ab72f690f33debaa51c7e88d9b969a0447a133eb2b99c985d9188cb2b
-> CACHED [backend 2/5] WORKDIR /app
-> [backend 3/5] COPY requirements.txt .
-> [frontend 2/2] COPY index.html /usr/share/nginx/html/index.html
-> [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt
-> [frontend] exporting to image
-> exporting layers
-> exporting manifest sha256:a7c3a5a470221611a70c2d18f1706d0752e3629f395650ee22ac7b3a5ef00bc5
-> exporting config sha256:067dbec16626a484bc20a61e276b041043f279254e7cf171f0dd6f8c9a0254f
-> exporting attestation manifest sha256:f6a6cc0182cb2818993575382b30fcaaa7aff7abe46e41a798f580929ed6f1
-> exporting manifest list sha256:d3361a24d85b0c2b4d4162f2f5964925f481cc4a187bef53061b9aad013
-> naming to docker.io/library/laskiewicz_martyna_projekt-frontend:latest
-> unpacking to docker.io/library/laskiewicz_martyna_projekt-frontend:latest
-> [frontend] resolving provenance for metadata file
-> [backend 5/5] COPY app.py .
-> [backend] exporting to image
-> exporting layers
-> exporting manifest sha256:b23b1f72c5c6e5adbfaed5fe082212e1a8cab82289f28208cc4f551d3e64e6d3
-> exporting config sha256:05927079efbc36eb20c09cd4948828f85eaf53a81764c9e7c420074b77fd
-> exporting attestation manifest sha256:c10793824459288f41af8bb0df2c0461a7227ef41e5799848a56461c285906d
-> exporting manifest list sha256:08e100e2483c5a1414153d059acc4116af6162949140c446f5a3afe3f5720
-> naming to docker.io/library/laskiewicz_martyna_projekt-backend:latest
-> unpacking to docker.io/library/laskiewicz_martyna_projekt-backend:latest
-> [backend] resolving provenance for metadata file
[*] Building 2/2
✓ laskiewicz_martyna_projekt-backend Built
✓ laskiewicz_martyna_projekt-frontend Built
PS C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt>
```

Sprawdzenie, czy aplikacja poprawnie uruchamia się w kontenerach.

Uruchamiamy aplikację poleceniem `docker compose up -d`. Kontenery powinny się uruchomić bez błędów. W terminalu powinny pojawić się logi aplikacji Flask informujące o starcie serwera na porcie 5000.

```
PS C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt> docker compose up -d
time="2025-12-14T14:48:50+01:00" level=warning msg="C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 3/3
✓ Network laskiewicz_martyna_projekt_default Created
✓ Container laskiewicz_martyna_projekt-backend-1 Started
✓ Container laskiewicz_martyna_projekt-frontend-1 Started
PS C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt>
```

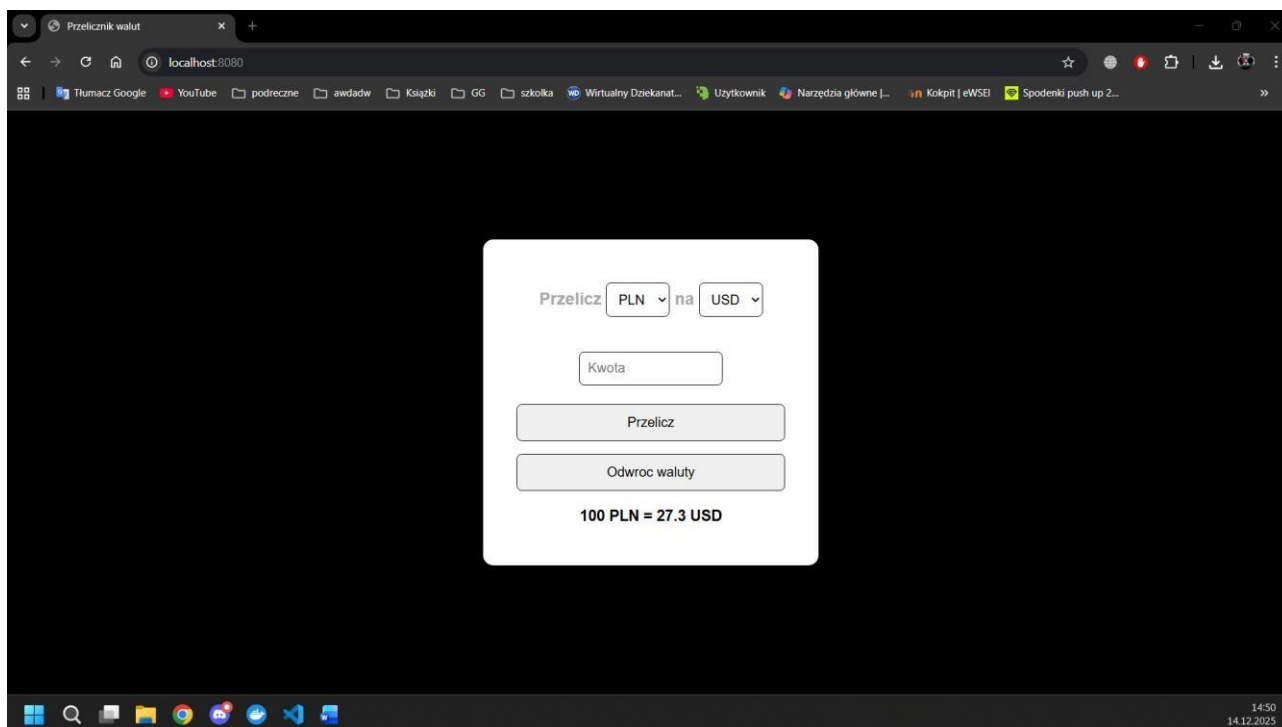
Sprawdzenie obrazów Dockera

Sprawdzamy, czy obrazy aplikacji zostały poprawnie utworzone poleceniem `docker compose images`. Na liście powinny się znajdować obrazy odpowiadające frontendowi oraz backendowi aplikacji.

```
PS C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt> docker compose images
time="2025-12-14T14:50:07+01:00" level=warning msg="C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
CONTAINER    REPOSITORY    TAG        PLATFORM    ID        SIZE    CREATED
laskiewicz_martyna_projekt-backend-1    laskiewicz_martyna_projekt-backend    latest    linux/amd64    60e100e2483c    48.1MB    2 minutes ago
laskiewicz_martyna_projekt-frontend-1    laskiewicz_martyna_projekt-frontend    latest    linux/amd64    d33e1a24d856    23MB    2 minutes ago
PS C:\Users\Tysia\Desktop\laskiewicz_Martyna_projekt>
```

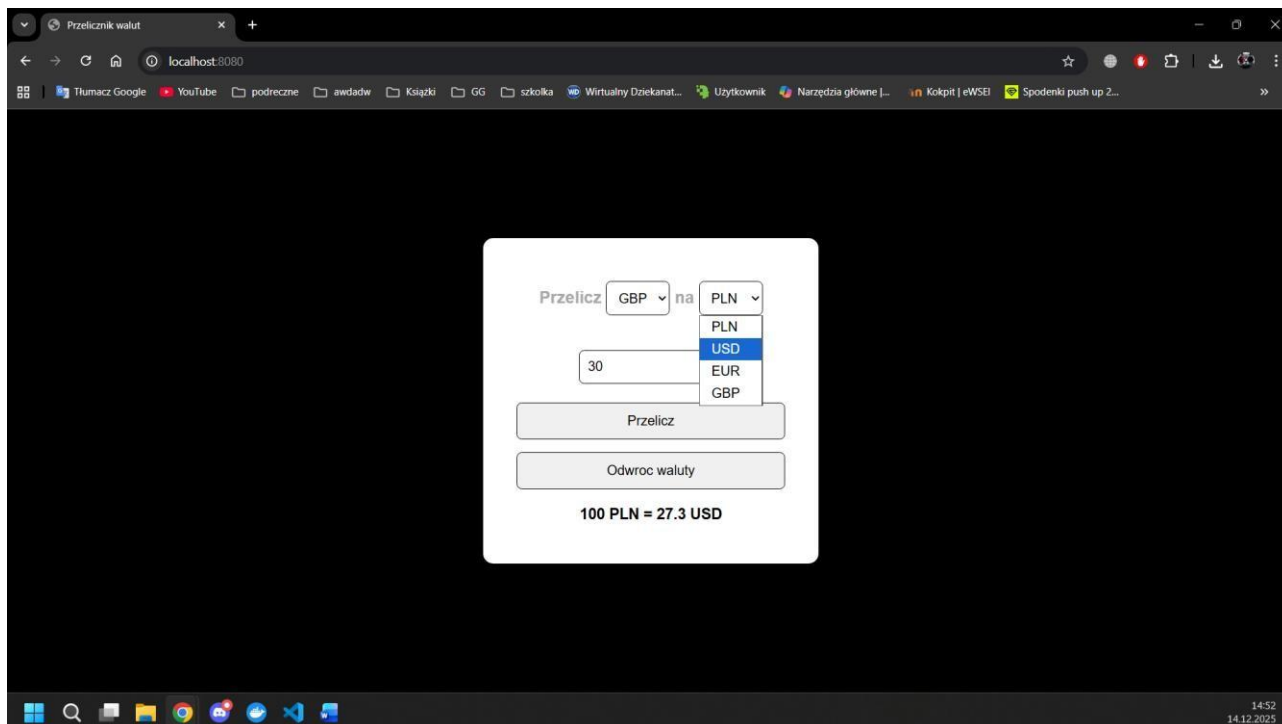
Sprawdzenie działania strony internetowej

Sprawdzamy, czy strona frontendowa poprawnie się wyświetla. Wchodzimy na stronę `http://localhost:8080`. Powinna wyświetlić się strona przelicznika walut z formularzem wyboru walut, polem do wpisania kwoty i dwoma przyciskami.



Sprawdzenie reakcji interfejsu użytkownika

Sprawdzamy, czy elementy interfejsu reagują na działania. Wybieramy waluty z list oraz wpisujemy dowolną kwotę. Listy rozwijane powinny poprawnie ustawiać waluty.



Test poprawnego przeliczenia waluty

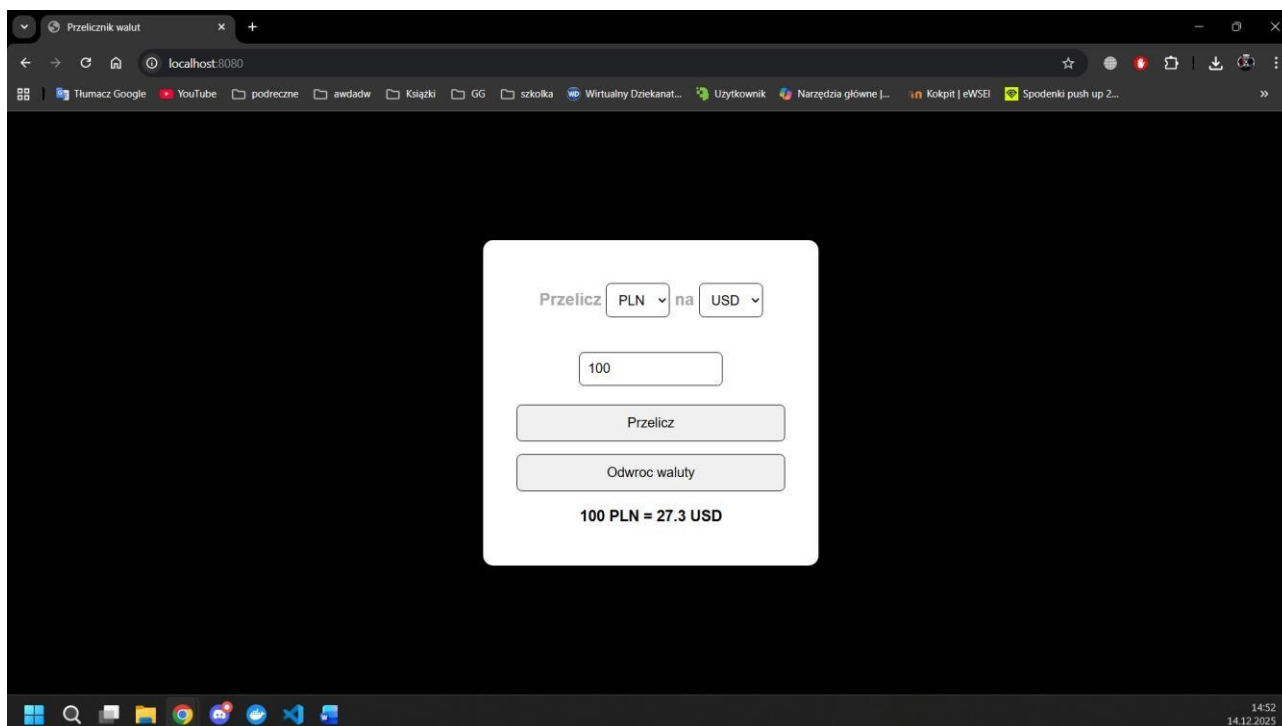
Sprawdzamy poprawność przeliczania walut według zdefiniowanych kursów, czyli:

- PLN: 1.0
- USD: 3.6624
- EUR: 4.2369
- GBP: 4.8375

Ustawiamy:

Przelicz *PLN* na *USD*
Kwota: *100*

Wynik powinien wynosić $100/3,6624=27,30$ USD

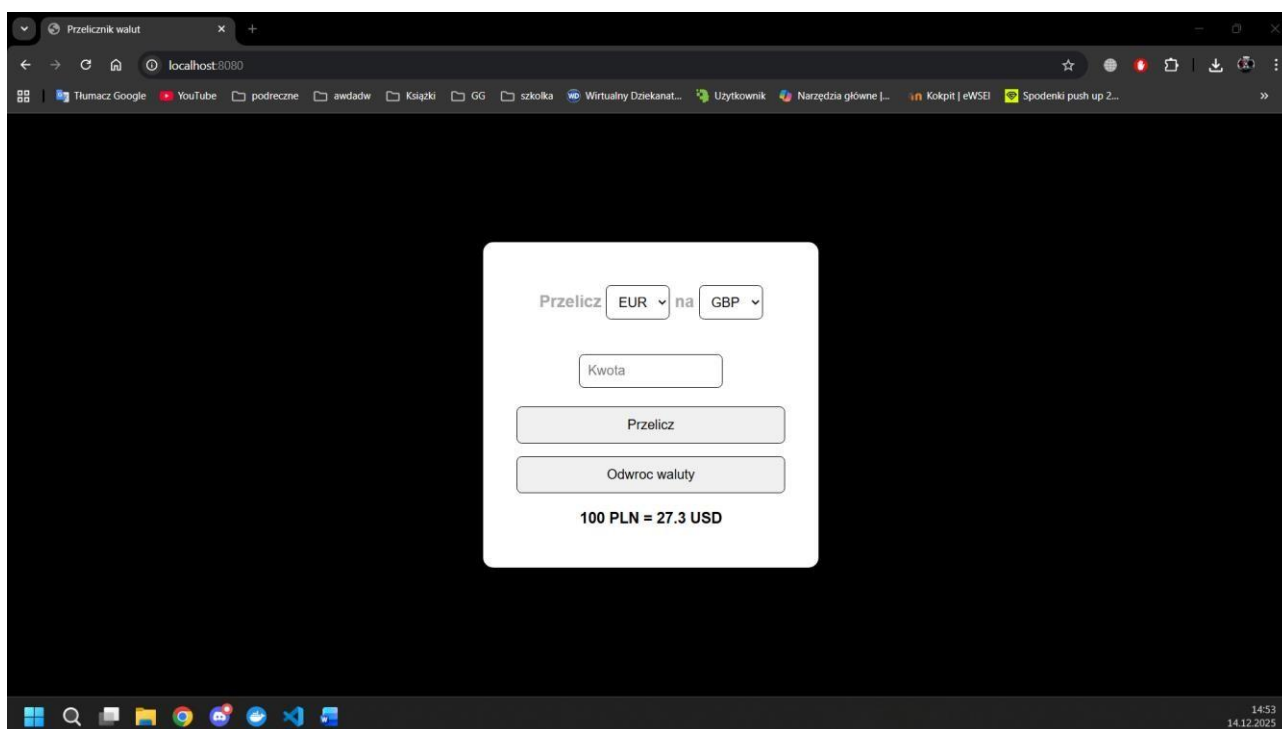


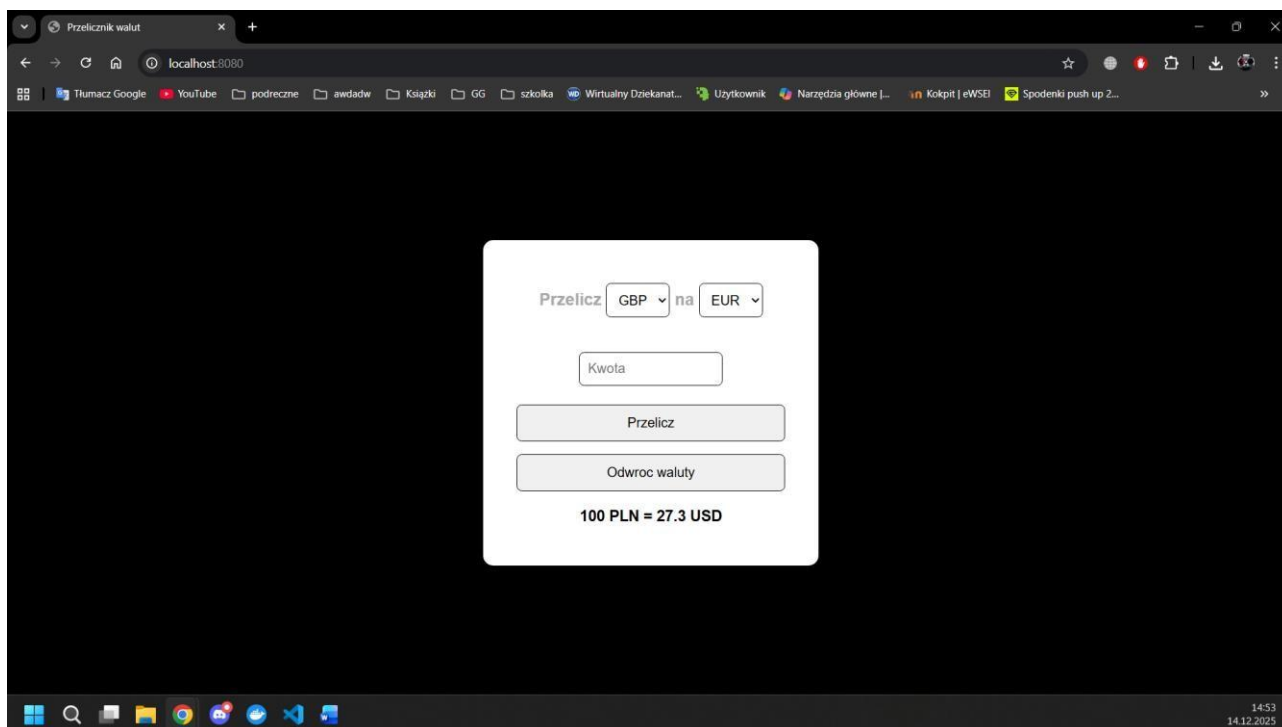
Sprawdzenie działania funkcji zamiany walut

Sprawdzamy poprawność działania funkcji odwracania walut. Wybieramy dwie inne waluty, np. EUR i GBP, a następnie klikamy guzik „Odwróć waluty”.

Wynik powinien być:

Przelicz *EUR* na *GBP* → Przelicz *GBP* na *EUR*





Sprawdzenie zabezpieczenia przed wpisaniem błędnej kwoty

Sprawdzamy, czy aplikacja poprawnie reaguje na wpisanie nieprawidłowej kwoty. Wpisujemy 1. wartość ujemną lub 0. / 2. zostawiamy puste pole / 3. próbujemy wpisać nie liczbę, losową literę.

1. Przelicz USD na PLN

Kwota: -130

Oczekiwany wynik: wyświetlenie napisu „błędna kwota”

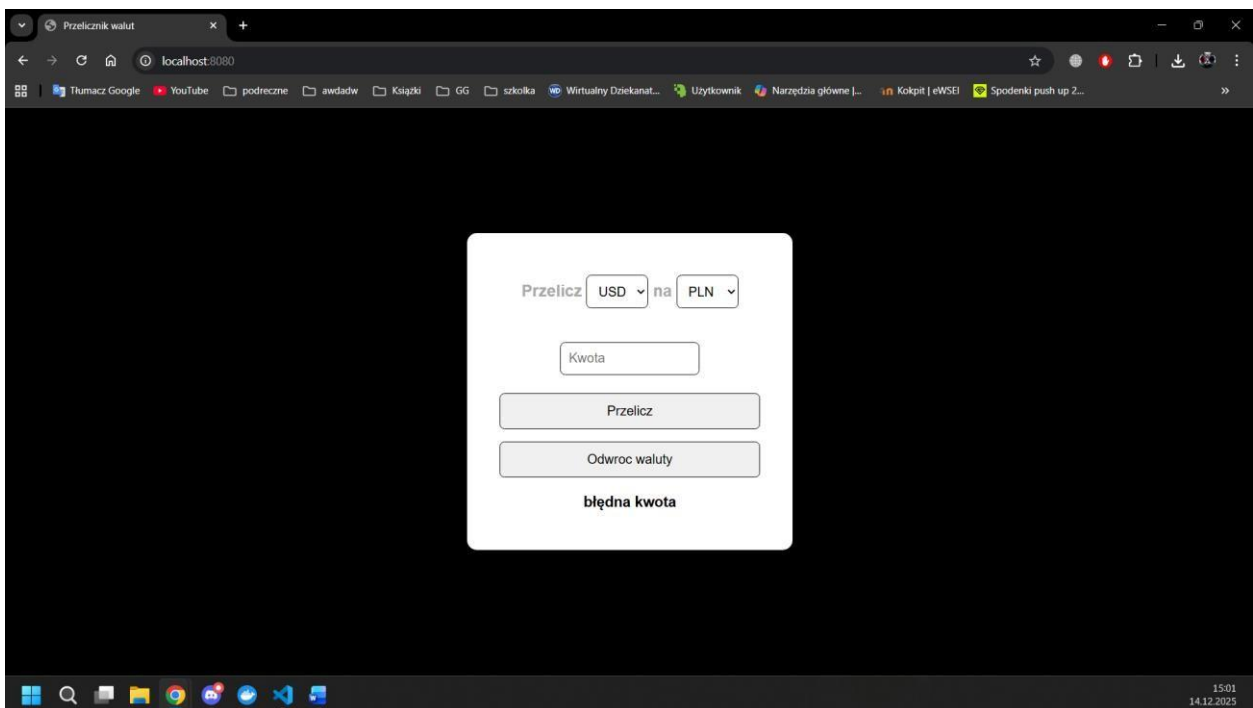
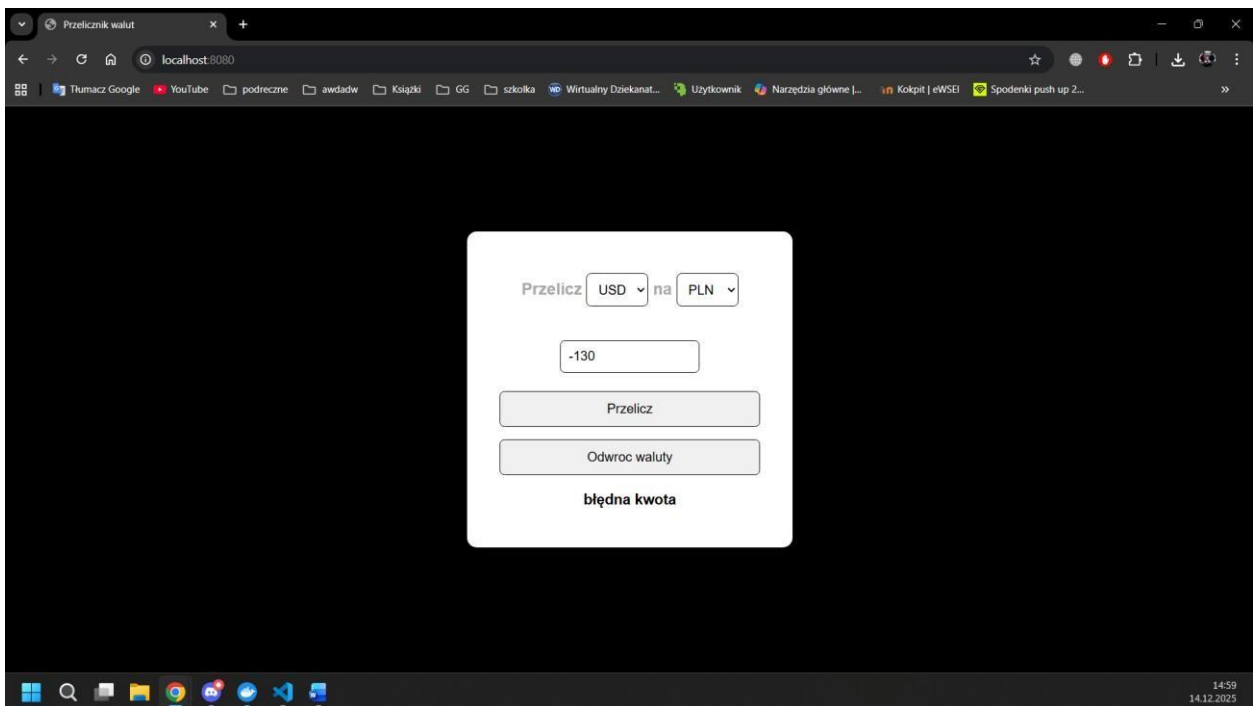
1. Przelicz USD na PLN Kwota:

Oczekiwany wynik: wyświetlenie napisu „błędna kwota”

2. Przelicz USD na PLN Kwota:

ddsadsa

Oczekiwany wynik: brak możliwości wpisania innych znaków niż liczbowe



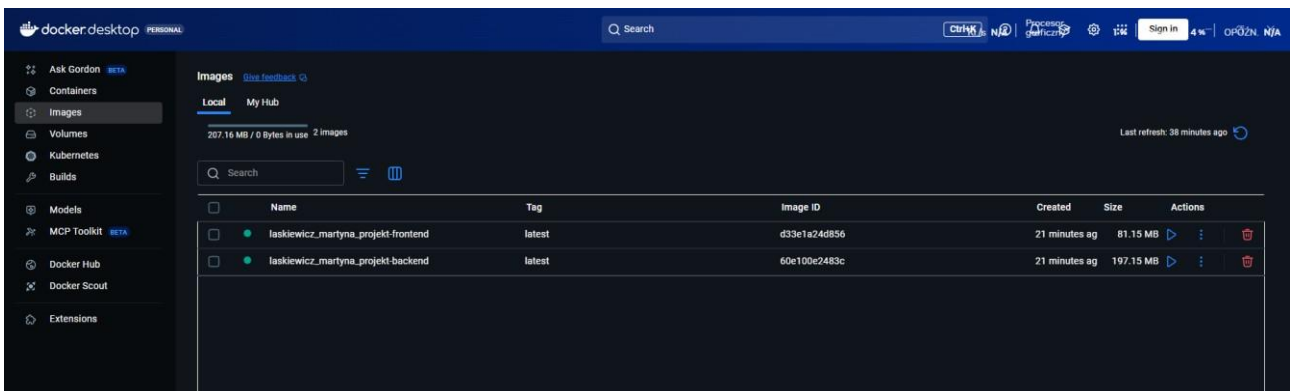
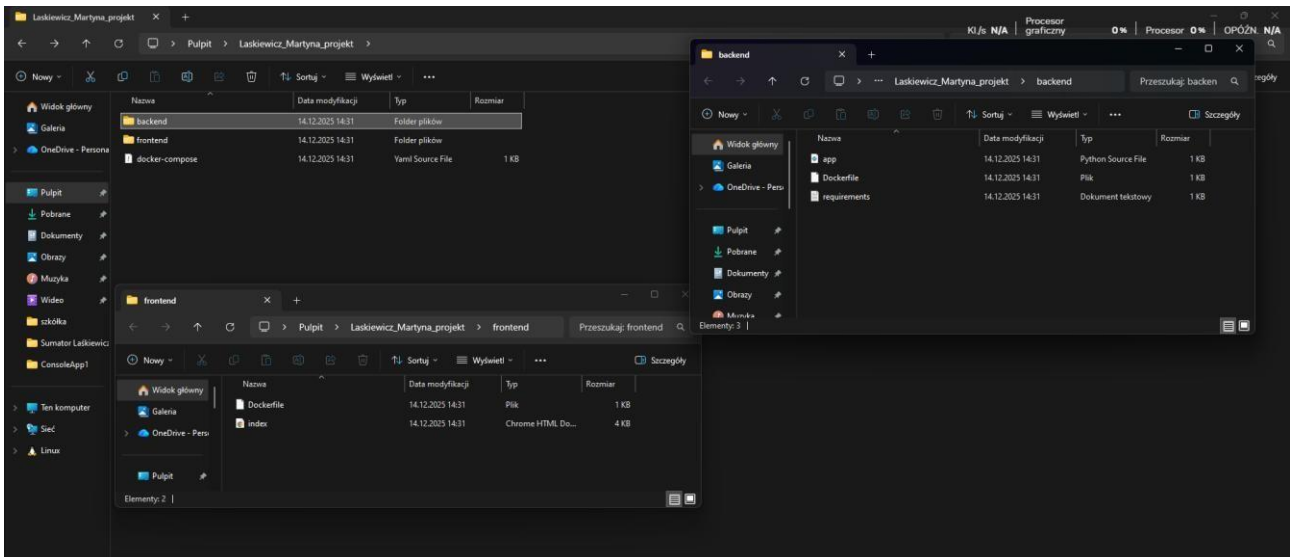
Pliki użyte w projekcie:

- **app.py** - główny plik aplikacji backendowej. Jest on odpowiedzialny za przeliczanie walut, obsługę HTTP oraz zwracanie danych w formacie JSON.
- **Dockerfile** - plik konfiguracyjny służący do budowania obrazu dla części backendowej projektu. Określa środowisko uruchomieniowe aplikacji.
- **docker-compose.yml** - plik konfiguracyjny Docker Compose. Uruchamia jednocześnie wszystkie elementy projektu.
- **requirements.txt** - plik tekstowy, zawierający listę usług wymaganych do poprawnego działania backendu.

- **index.html** – frontend aplikacji, czyli nasza strona internetowa. Odpowiada za interfejs, umożliwia wybór walut, wprowadzenie kwoty oraz komunikację z backendem za pomocą JS.

```
PS C:\Users\Tysia\Desktop\Laskiewicz_Martyna_projekt> tree /f
Folder PATH listing
Volume serial number is 224A-E0B5
C:.
├── docker-compose.yml
├── backend
│   ├── app.py
│   ├── Dockerfile
│   └── requirements.txt
└── frontend
    ├── Dockerfile
    └── index.html

PS C:\Users\Tysia\Desktop\Laskiewicz_Martyna_projekt>
```



Celem projektu było wykonanie prostej aplikacji internetowej wykorzystującej architekturę mikroserwisów do stworzenia usługi webowej. Udało mi się to dzięki stworzeniu backendowego mikroserwisu w technologii Flask oraz warstwy frontendowej opartej na HTML, CSS i JS.

Aplikacja umożliwia poprawne przeliczanie walut na podstawie zdefiniowanych kursów oraz zapewnia prawidłową komunikację między wszystkimi warstwami aplikacji.

W dalszym etapie możliwa jest rozbudowa aplikacji o dodatkowe funkcjonalności, typu pobieranie aktualnych kursów walut, zapisywanie historii przeliczeń czy dodanie większej ilości walut.

Literatura:

- Docker Inc., *Docker Overview*. <https://docs.docker.com/get-started/docker-overview>
- Docker Inc., *What is a container?*. <https://www.docker.com/resources/what-container>
- Pallets Projects, *Flask – dokumentacja frameworka*. <https://flask.palletsprojects.com/en/stable>
- Wikipedia, *Docker (oprogramowanie)*. [https://pl.wikipedia.org/wiki/Docker_\(oprogramowanie\)](https://pl.wikipedia.org/wiki/Docker_(oprogramowanie))
- Fowler M., *Microservices*. <https://martinfowler.com/articles/microservices.html>
- Mozilla Developer Network, *HTML – dokumentacja*. <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Wikipedia, *HTML*. <https://pl.wikipedia.org/wiki/HTML>
- *Introducing JSON*. <https://www.json.org/json-en.html>
- Docker Inc., *Docker Compose – dokumentacja*. <https://docs.docker.com/compose>
- Wikipedia, *Mikroserwisy*. <https://pl.wikipedia.org/wiki/Mikroserwisy>
- UProgramisty.pl, *Docker Compose – zarządzanie wieloma kontenerami*. <https://uprogramisty.pl/docker-compose-zarzadzanie-wielomakontenerami/>